Dayflow HRMS
API Contract Specification

Backend–Frontend Integration Document

1 Purpose of this Document

This document defines the complete API contract for the Dayflow Human Resource Management System (HRMS). It serves as a single source of truth for backend and frontend development.

All data exchange between frontend and backend MUST follow this specification exactly.

2 System Overview

- Architecture: API-first
- Backend: FastAPI
- Frontend: React
- Database: SQLite (MVP)
- Data format: JSON
- Authentication: JWT (Bearer Token)

3 User Roles

- **EMPLOYEE**: Regular employee with limited access
- **ADMIN**: HR/Admin with management and approval privileges

4 Authentication Rules

- All APIs except `/auth/login` require authentication
- JWT token must be passed in HTTP header:

```
Authorization: Bearer <JWT_TOKEN>
```

5 API Summary

Feature	Endpoint	Role

Login	POST /auth/login	All
Get Current User	GET /auth/me	All
View Own Profile	GET /users/me	Employee
Update Own Profile	PUT /users/me	Employee
View All Employees	GET /admin/users	Admin
Check-in Attendance	POST /attendance/check-in	Employee
Check-out Attendance	POST /attendance/check-out	Employee
View Own Attendance	GET /attendance/me	Employee
View All Attendance	GET /admin/attendance	Admin
Apply Leave	POST /leave/apply	Employee
View Own Leave Requests	GET /leave/me	Employee
View All Leave Requests	GET /admin/leave	Admin
Approve/Reject Leave	PUT /admin/leave/id	Admin
View Own Payroll	GET /payroll/me	Employee

6 Detailed API Specifications

6.1 Authentication APIs

6.1.1 POST /auth/login

Purpose: Authenticate user and issue JWT token.

Request Body:

```
{
  "email": "user@company.com",
  "password": "password123"
}
```

Response:

```
{
  "access_token": "jwt-token-string",
  "role": "EMPLOYEE"
}
```

6.1.2 GET /auth/me

Purpose: Fetch currently authenticated user details.

Response:

```
{
  "id": 1,
  "employee_id": "EMP001",
  "name": "John Doe",
  "email": "john@company.com",
  "role": "EMPLOYEE"
}
```

7 Employee Profile APIs

7.0.1 GET /users/me

Purpose: Fetch employee's own profile.

Response:

```
{
  "name": "John Doe",
  "phone": "9999999999",
  "address": "Ahmedabad",
  "job_title": "Software Engineer",
  "salary": 50000
}
```

7.0.2 PUT /users/me

Purpose: Update editable employee fields.

Request Body:

```
{
  "phone": "8888888888",
  "address": "Surat"
}
```

8 Attendance Management APIs

8.0.1 POST /attendance/check-in

Purpose: Record employee check-in time.

Response:

```
{
  "message": "Checked in successfully",
  "time": "09:30"
}
```

8.0.2 POST /attendance/check-out

Purpose: Record employee check-out time.

Response:

```
{
  "message": "Checked out successfully",
  "time": "18:05"
}
```

8.0.3 GET /attendance/me

Purpose: View own attendance history.

Response:

```
[
  {
    "date": "2026-01-03",
    "check_in": "09:30",
    "check_out": "18:05",
    "status": "Present"
  }
]
```

8.0.4 GET /admin/attendance

Purpose: Admin view of all attendance records.

Response:

```
[
  {
    "employee_name": "John Doe",
    "date": "2026-01-03",
    "status": "Present"
  }
]
```

9 Leave Management APIs

9.0.1 POST /leave/apply

Purpose: Submit a leave request.

Request Body:

```
{
  "start_date": "2026-01-05",
  "end_date": "2026-01-06",
  "leave_type": "Sick",
  "reason": "Fever"
}
```

9.0.2 GET /leave/me

Purpose: View own leave requests.

Response:

```
[
  {
    "id": 1,
```

```
    "leave_type": "Sick",
    "status": "Pending"
  }
]
```

9.0.3 GET /admin/leave

Purpose: Admin view of all leave requests.

9.0.4 PUT /admin/leave/id

Purpose: Approve or reject a leave request.
Request Body:

```
{
  "status": "Approved",
  "comment": "Approved by HR"
}
```

10 Payroll API

10.0.1 GET /payroll/me

Purpose: Employee view of own salary (read-only).
Response:

```
{
  "basic": 40000,
  "allowances": 10000,
  "deductions": 2000,
  "net_salary": 48000
}
```

11 Feature Call Order (Flow)

11.1 Employee Flow

1. POST /auth/login
2. GET /auth/me
3. GET /users/me
4. POST /attendance/check-in
5. POST /attendance/check-out
6. POST /leave/apply
7. GET /leave/me

11.2 Admin Flow

1. POST /auth/login
2. GET /admin/users
3. GET /admin/attendance
4. GET /admin/leave
5. PUT /admin/leave/id

12 Final Notes

- Frontend MUST NOT access database directly
- Backend MUST enforce role-based access
- API responses must strictly follow defined formats
- Any deviation must be discussed and documented